Software for Embedded Systems

Contents

1. Linux for Embedded Systems 2
   1.1. Linux Kernel 2
2. Tasks & Task Management 2
   2.1. Tasks 2
      2.1.1. Thread 2
3. Shared Variables 3
   3.1. Ping Pong Buffer 3
   3.2. Ring Buffer 4
   3.3. MailBox 4
4. Task Synchronization 5
   4.1. Active Resource 5
   4.2. Passive Resource 5
   4.3. Requirements 5
   4.4. Solution 5
      4.4.1. Flags 5
      4.4.2. Token Passing 6
      4.4.3. Manage using interrupts 6
5. Semaphores 6
   5.1. Spin Locking and Busy Waiting 7
   5.2. Semaphores - Counting 7
   5.3. Implementation 7
6. Example - Mars Rover 7
   6.1. Semphores: 8
7. Bounded Buffer Problem 8

1. Linux for Embedded Systems

- A kernel for RTES - `Prempt RT`.
- Linux has 4 components:
  - Kernel
  - Shared Libraries – can be called by any program
  - System command & Utilities
  - User apps
- **Kernel** and **Shared Libraries** are in **Kernel Space**.
- **System Command** and **User apps** are in **User Space**.

1.1. Linux Kernel

- Includes **Daemon** proceses.
- **Init** is a process that initiates Daemon processes. It always has **PID 1**.
- `systemctl list-units --tyye =service`.

2. Tasks & Task Management

Embedded Systems - reactive in nature.

> **Note:**
> Jitter - uncertain
> Delay - certain

2.1. Tasks

- Embedded program - collection of firmware modules.
- Firmware modules executes - Process/Task.
- Control ensures that tasks execution satisfies set of timing constraints - **Real Time**

Process is allocated following resources:

- **CPU**
  - PC
  - SP
  - General Purpose Registers
  - Status/Flag Registers
- **Stack/Heap**
- **TCB creation**

Persistence: Duration between when task enters to termination.

2.1.1. Thread

- Lightweight process
- can share resources
- doesn't require separate address space

Thread Types

- Interleaved - 3th
- Blocked
- Simultaneous - 4th

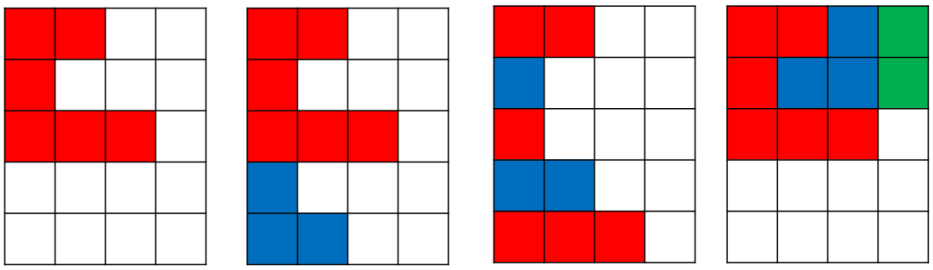

Figure 1: Thread Types

Note: Hyperthreading(Intel) and **Simultaneous**(ARM) are the same thing.

3. Shared Variables

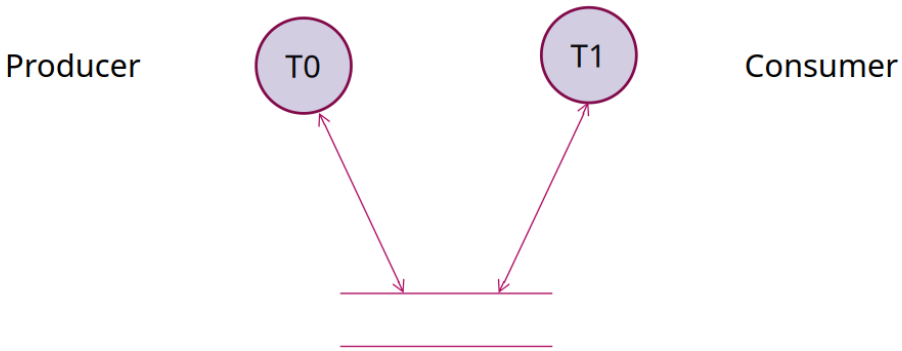

Figure 2: Shared Variables

- Producer - checks for not full - `Bool Full();`
- Consumer - checks for not empty - `Bool Empty();`

Atomicity - Either the task will complete or won't start.

3.1. Ping Pong Buffer

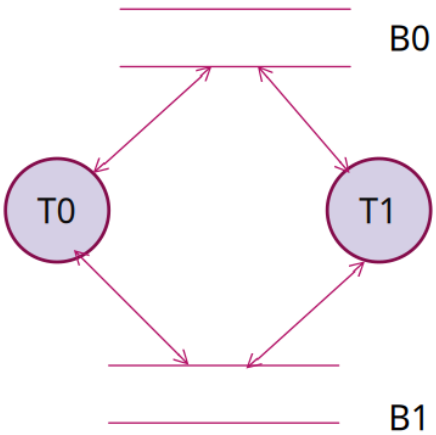

Figure 3: Ping Pong Buffer

- When producer is writing into `B0`, consumer is reading from `B1` or vice versa.
- So one buffer at a given time will be used by either producer or consumer.

3.2. Ring Buffer

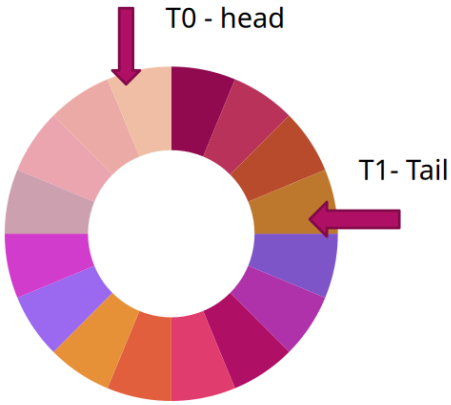

Figure 4: Ring Buffer

- `T0` and `T1` will start from the same buffer.
- `T0` is producer and `T1` is consumer.

3.3. MailBox

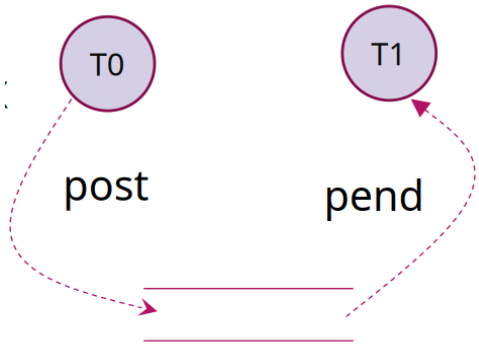

Figure 5: MailBox

- `Producer` or `post` will send message.
- `Consumer` or `pend` will recieve message.

> **Note:**
> - For **streaming**, Direct Communication is used.
> - For **controlling**, Indirect communication is used.

4. Task Synchronization

Producer	Consumer
while(1)	while(1)
if not full	if not empty
add item	get item
inc count	dec count
else	else
wait for space	wait for item
endwhile	endwhile

Figure 6: Co-operating Tasks

- `count` is shared variable. We need to use mutex.

4.1. Active Resource

Active resource ends up being consumed.

- Examples:
  - CPU
  - Memory (depending on how it is accessed)

4.2. Passive Resource

- Memory
- Bus

4.3. Requirements

- Mutial exlusion
- Deadlock
- Bounded Waiting - n tasks can run before the waiting of the current task stops.

4.4. Solution

4.4.1. Flags

- Each Task has a flag
- Atomic procedure await

Producer	Consumer
While(1)	While(1)
if not full	if not empty
add item	get item
await(!T1Flag) {To Flag= true}	await(!ToFlag) {T1 Flag = true}
inc count	dec count
ToFlag = False	T1Flag = False
else	else
wait for space	wait for item
endwhile	endwhile

Figure 7:

Cons

- Doesn't work with multiple tasks.

4.4.2. Token Passing

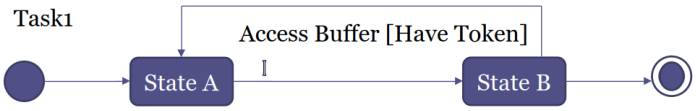

Figure 8:

Cons

- Starvation of lower proiority task.

4.4.3. Manage using interrupts

- Disallow interrupts in the critical section.

5. Semaphores

- Two purposes:
  - Threads are going to execute in particular order.
  - Mutual Exclusion
- wait - `p(s)` -> set flag
- signal - `v(s)` -> reset flag

Producer	Consumer
{	{
....	....
wait(s)	wait(s)
critical section	critical section
signal(s)	signal(s)
....	....
}	}

Figure 9: Semaphores - Critical Resources

wait(s)	signal(s)
{	{
while(s);	s = false;
s = true;	}
}	

Figure 10: Semaphores

5.1. Spin Locking and Busy Waiting

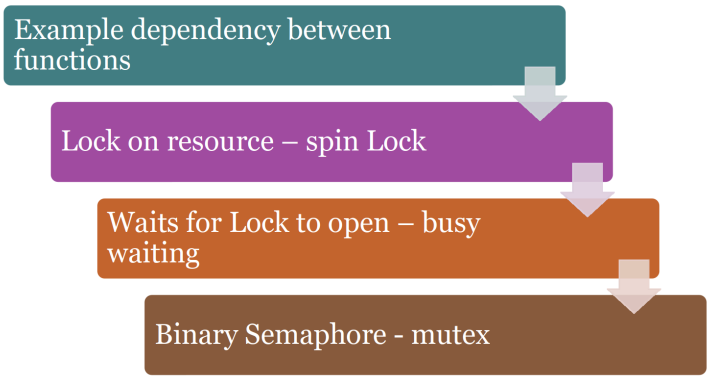

Figure 11: Spin Locking and Busy Waiting

- **It is given in Textbook that `Binary Semaphores` are `Mutex`. But that is wrong.**
  - Mutex has ownership.
  - Linux uses **futexes**.

5.2. Semaphores - Counting

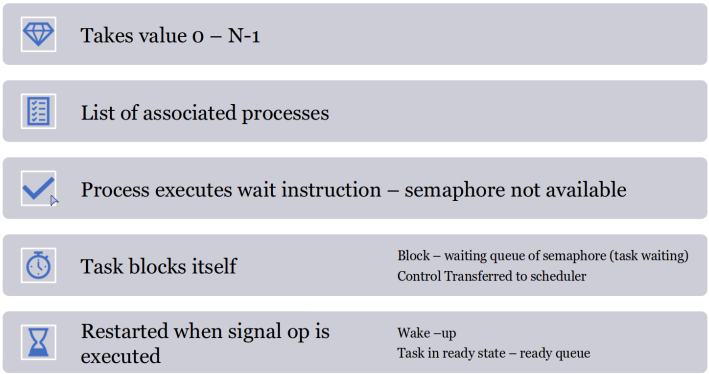

Figure 12: Counting Semaphores

5.3. Implementation

```
typedef struct {
    int value;
    queue tlist;
} semaphore;
```

6. Example - Mars Rover

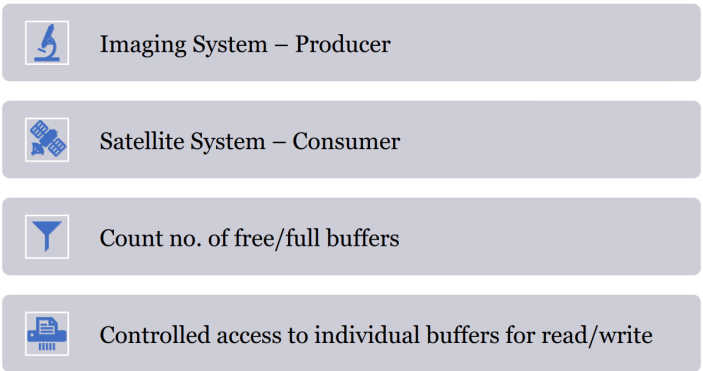

Figure 13: Requirements

6.1. Semphores:

- `mutex(1)`
- `empty(n-1)`
- `full(0)`

Camera

```
while(1)
  ....
  produce an item
  ....
  wait(empty);
          wait(mutex);
          ....
           add item to
  buffer;
          ....
          signal(mutex);
  signal(full);
  ....
endwhile
```

Satellite

```
while(1)
  wait(full);
          wait(mutex);
          ....
           remove item from
                  buffer;
          ....
          signal(mutex);
  signal(empty);
  ....
  consume  item
  ....

endwhile
```

Figure 14: Consumer & Producer

wait(empty)

```
sem_wait (semaphore *S)
 {
          S->value--;
          if (S->value < 0) {
          add this process to
          S->tlist;
          block();
  }
```

signal(empty)

```
sem_signal (semaphore *S) {
          S->value++;
          if (S->value <= 0) {
          remove thread t
  from    S->tlist;
          wakeup(t);
  }
}
```

Figure 15: Semaphores

7. Bounded Buffer Problem